Case Study of Security Assurance and Compliance Verification in AI-Generated Software Systems

Name of Authors:
Abhishek Choudhary, Abhayjeet Singh, Chirag, Simran Jaswal
Chitkara University, Punjab, India
Email: abhishek1161.be22@chitkara.edu.in,
abhayjeet31.be22@chitkara.edu.in, chirag1463.be22@chitkara.edu.in, simran2383.be22@chitkara.edu.in

***Abstract*— The advent of large language model (LLM)-powered coding assistants in professional software development has opened new security challenges that are beyond the capabilities of traditional quality assurance workflows. While productivity gains are real, recent evidence suggests that AI-generated code often contravenes secure coding practices without failing functional tests, making such vulnerabilities undetectable with traditional verification processes. This paper describes a controlled study of the security of AI-generated code using a multi-stage vulnerability detection pipeline, integrating static application security testing (SAST) with manual code review. Code artifacts are compared against the OWASP Top 10 (2021) vulnerability list. Results identify patterns of vulnerabilities in injection, broken access control, and cryptographic failure vulnerabilities, and have implications for the design and placement of security gates in AI-augmented software development processes. The paper closes with practical insights and recommendations for future work.**

***Index Terms*— AI-generated code; secure coding; OWASP Top 10; static analysis; large language models; software security; vulnerability detection.**

I. INTRODUCTION

Code generation, once considered a specialised feature of template engines and bespoke compilers, has taken a step change with the advent of large language models (LLMs) that have been trained on billions of lines of publicly available source code. Software systems like GitHub Copilot, Amazon CodeWhisperer and Anthropic Claude are now integral to professional software development processes, regularly generating entire function bodies, data-access layers, and authentication subsystems in response to natural language prompts. Professional surveys show that developer productivity has improved, and the growth in tool adoption indicates their impact on the software development process will increase significantly in future.

Productivity and security are not synonymous objectives. A growing body of empirical evidence indicates that LLMs optimise for functional correctness—the capacity of generated code to compile, execute, and pass unit tests—rather than conformance with established secure coding standards. The consequence is a systematic divergence between apparent correctness and actual security posture: code that satisfies all stated functional requirements may simultaneously introduce latent vulnerabilities that escape detection by conventional testing workflows.

The systemic risk dimension distinguishes AI-generated vulnerabilities from their human-authored counterparts. If a particular vulnerability class is underrepresented in a model's training signal, instances of that weakness may be reproduced across thousands of codebases simultaneously. This multiplicative property elevates the importance of systematic, reproducible evaluation methodologies capable of detecting such patterns before they propagate to deployed systems.

The OWASP Top 10 [1] provides a widely recognised, empirically grounded taxonomy of the ten most consequential web-application security risk categories and serves as the evaluative reference for the present study. Its 2021 edition is notable for introducing Insecure Design (A04) as a structural risk category distinct from implementation defects, a distinction that carries particular significance in the context of AI code generation, where design-level decisions are implicitly embedded in model outputs.

This paper reports the design and outcomes of a controlled experimental evaluation of AI-generated code security. The study yields the following principal contributions:

1) A reproducible experimental protocol for eliciting security-sensitive AI-generated code across representative development scenarios.
2) A layered vulnerability detection pipeline integrating SAST tooling with expert manual review.
3) An annotated vulnerability dataset mapped to OWASP Top 10 (2021) categories, severity ratings, and remediation outcomes.
4) Practitioner-oriented recommendations for positioning security gates within AI-augmented development workflows.

II. LITERATURE SURVEY

A. Foundational Context

Security of AI-Generated Code. In the wake of the commercial release of GitHub Copilot in 2021, there was a notable spike in interest with respect to the security implications of AI-assisted programming. Pearce et al. [2] carried out an initial security analysis, developing forty

programming scenarios based on CWE entries covering memory management, cryptography, and web input processing. The results indicated that approximately 40% of generated code contained at least one exploitable vulnerability.

Perry et al. [3] tackled this from a human factors viewpoint, conducting a controlled experiment to determine whether developers aided by AI produced more or less secure code than those who were not. In certain categories of tasks, developers aided by AI were significantly less secure, and notably exhibited greater subjective confidence in the security of their code.

Tihanyi et al. [7] measured a number of different LLMs against a standardised set of prompts designed to induce different vulnerabilities. Correctness and conformance to security rules were found to be largely independent, with each model exhibiting characteristic CWE tendencies that remained stable across prompt variations.

OWASP Top 10 as an Evaluative Reference. The OWASP Top 10 list represents a periodically revised consensus list of the most impactful web application security risk categories. The 2021 release formally separated Insecure Design (A04) as a top-level category, distinct from implementation-level defects. This distinction is analytically important when assessing AI-generated code, since LLMs make implicit design-level decisions that may be insecure at the architectural level without any syntactically identifiable flaw. Categories A01, A02, A03, and A07 received the most significant analysis in the present study based on their prevalence in the preliminary scanning process.

Static Analysis in Secure Development Pipelines. Static Application Security Testing (SAST) instruments analyse source code without executing the program, detecting security-relevant patterns through rule matching, data flow, or taint propagation. The three SAST instruments used in this study—Semgrep, Bandit, and pip-audit—have different analytical capabilities. Semgrep offers cross-language rule support; Bandit offers Python-specific vulnerability detection with rules mapped to CWEs; pip-audit checks declared dependency trees for known CVEs. All three are established in the literature and industry, making this study comparable to prior works. The main limitation of SAST tools is the production of false positives and false negatives, addressed here by a layered detection pipeline supplemented with structured manual review.

B. Related Work

Research on the security of AI-generated code spans three thematically distinct clusters: empirical assessments of model outputs, the application of formal vulnerability taxonomies to AI code corpora, and the design of automated detection pipelines for SDLC integration.

Khoury et al. [5] evaluated ChatGPT-generated code across five programming languages and five targeted vulnerability classes, observing that while outputs were generally syntactically well-formed, they routinely omitted input validation, authorisation enforcement, and cryptographically sound primitive selection. Siddiq and Santos [8] applied CWE-based classification to a corpus of Copilot-generated code, identifying recurring weakness patterns strongly correlated with OWASP Top 10 categories, with injection and authentication failure classes appearing most persistently. Asare et al. [9] confirmed that model outputs could replicate historically documented vulnerability patterns from training data, motivating the inclusion of a manual review stage in detection pipelines intended for research use.

Taken collectively, the reviewed literature demonstrates that AI coding assistants pose security risks that are systematic and model-specific, and for which current software development practices are insufficient. This study addresses methodological limitations in existing research by incorporating a detection pipeline designed to control for both false positive and false negative error types.

III. METHODOLOGY

The research methodology is structured as a four-phase empirical study: (1) prompt corpus construction, (2) code generation and collection, (3) multi-layer vulnerability detection, and (4) quantitative analysis and pattern classification. Each phase is described in sufficient detail to enable independent replication.

A. Research Design Overview

The study adopts a controlled experimental design in which the type, count, and severity of security defects present in generated outputs serve as the dependent variables, and the AI code generation tool serves as the independent variable. A within-subjects design was selected so that identical prompts could be issued to each tool, controlling for task difficulty as a potential confounding factor. The unit of analysis is a single generated Python code artifact corresponding to a single task prompt.

B. Prompt Corpus Construction

A corpus of thirty task prompts was constructed to represent functional scenarios commonly encountered in web application development. Prompts were distributed evenly across three domains: (i) User Authentication, covering registration, login, session management, password reset, and role-based access control; (ii) Data Management, covering database read/write operations, search filtering, and report export; and (iii) Data Transmission, covering REST API endpoint creation, file upload handling, and inter-service communication.

Prompts were deliberately composed in natural language without explicit security requirements, replicating conditions in which a developer describes desired functionality without specifying defensive constraints. Each prompt was peer-reviewed by two co-authors prior to inclusion; ambiguous items were revised through a structured clarification protocol.

C. Code Generation and Collection

The thirty prompts were submitted to three AI code generation systems: GPT-4o (OpenAI), GitHub Copilot (Microsoft), and Claude 3.5 Sonnet (Anthropic). For GPT-4o and Claude 3.5 Sonnet, prompts were submitted programmatically via their respective public APIs using a Python script (available in the project repository). Each prompt was issued in a fresh session with no prior context, and model temperature was set to 0 (greedy decoding) to minimise output non-determinism. For GitHub Copilot, which does not expose a public completion API, prompts were submitted manually through the VS Code extension in an isolated workspace; the generated outputs were immediately saved without any post-hoc modification. This yielded 90 code artifacts in total (30 per tool), all written in Python.

Generated artifacts were stored in a structured directory layout organised by tool name and prompt identifier. No researcher modified any artifact after collection. All metadata—including tool name, prompt identifier, programming language, and collection timestamp—were recorded in a companion JSON file.

D. Multi-Layer Vulnerability Detection Pipeline

Vulnerability detection was conducted using a three-stage pipeline designed to provide complementary coverage of both syntactic and semantic weakness patterns.

1) Automated Static Analysis: All 90 artifacts were first scanned using Bandit v1.7.5, a Python-specific SAST tool whose rules map directly to CWE entries. Bandit was configured with its full default rule set and HIGH confidence filtering to reduce low-signal findings. Semgrep v1.45 was then applied for cross-validation using the publicly available OWASP Top 10 community rule set (`p/owasp-top-ten`). Outputs from both tools were normalised into a common structured format capturing the artifact identifier, rule identifier, CWE number, OWASP Top 10 category, line number, severity (LOW / MEDIUM / HIGH / CRITICAL), and confidence level.

2) Dependency and Configuration Analysis: Artifacts that imported third-party packages were subjected to dependency scanning using pip-audit v2.6.1, which checks declared package names against the Python Packaging Advisory Database (PyPA) to surface known CVEs in transitive dependencies. Configuration-level issues—including Flask debug mode being enabled, permissive CORS policy declarations, and disabled SSL certificate verification—were detected through a custom Semgrep rule set of 18 rules authored by the research team and validated against a held-out set of 20 labelled examples.

3) Manual Expert Review: A stratified sample of 22 artifacts (approximately 25% of the corpus) was selected for manual review, with proportional representation across all three tools and all three task domains. Two reviewers with formal training in application security independently examined each sampled artifact using a structured checklist adapted from the OWASP Code Review Guide v2.0 [6].

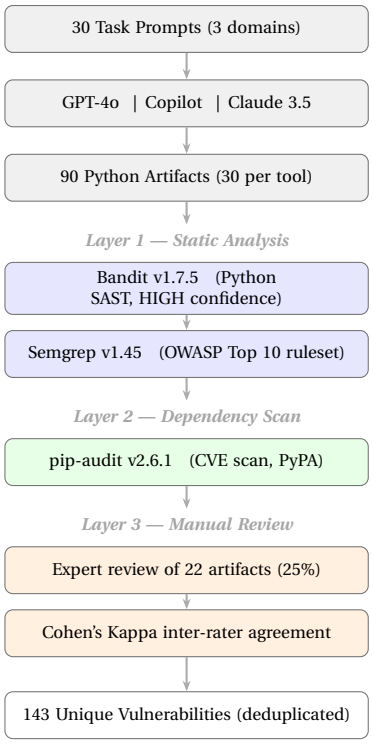

Fig. 1. Three-layer vulnerability detection pipeline applied to 90 AI-generated Python artifacts.

The checklist addressed seven categories: input validation, output encoding, authentication and session management, access control, cryptographic storage, error handling, and logging and monitoring. Inter-rater agreement was computed using Cohen's Kappa [10]; artifacts with Kappa below 0.70 were escalated to a third reviewer for adjudication. SAST tool findings on the reviewed sample were compared against expert judgements to compute the false positive rate.

IV. EXPERIMENTAL SETUP

A. Hardware and Software Environment

All analyses were conducted on personal laptops used by the research team running Windows 11 and macOS Ventura. No dedicated server infrastructure or containerisation was required; all tools were installed directly via pip into a Python 3.11 virtual environment, making the setup reproducible on any standard development machine. Table I summarises the tool versions and configurations used throughout the study.

B. Benchmark Dataset

The benchmark dataset comprised 90 Python code artifacts: 30 generated by GPT-4o, 30 by GitHub Copilot, and 30 by Claude 3.5 Sonnet. Each set of 30 contained 10 artifacts per task domain. Table II presents the full distribution.

TABLE I

TOOL CONFIGURATION FOR VULNERABILITY DETECTION PIPELINE

Tool	Ver.	Role / Configuration
Bandit	1.7.5	Python SAST; full ruleset, HIGH confidence
Semgrep	1.45.0	Multi-lang SAST; OWASP Top 10 ruleset
pip-audit	2.6.1	CVE scan; full dependency tree
CodeClimate	CLI 0.95	Supplementary SAST; security + complexity

TABLE II

ARTIFACT DISTRIBUTION ACROSS TOOLS AND TASK DOMAINS

AI Tool	Auth.	Data Mg.	Data Tx.	Total
GPT-4o	10	10	10	30
GitHub Copilot	10	10	10	30
Claude 3.5 Sonnet	10	10	10	30
Total	**30**	**30**	**30**	**90**

C. Vulnerability Severity Classification

Vulnerabilities were classified using a four-tier scheme aligned with CVSS v3.1 [4] base score ranges. CRITICAL and HIGH findings were treated as requiring immediate action for the purposes of computing the Critical Vulnerability Rate (CVR) metric. Table III summarises the scheme.

TABLE III

SEVERITY CLASSIFICATION SCHEME (CVSS V3.1)

Severity	CVSS Score	Action Required
CRITICAL	9.0–10.0	Immediate remediation; block deployment
HIGH	7.0–8.9	Remediation required before deployment
MEDIUM	4.0–6.9	Fix in next planned sprint
LOW	0.1–3.9	Address in scheduled maintenance

D. Metrics and Evaluation Criteria

The following metrics were computed per tool and per task domain: Vulnerability Density (VD), defined as unique vulnerabilities per 100 lines of code; OWASP Coverage Score (OCS), the fraction of OWASP Top 10 categories represented in findings; Critical Vulnerability Rate (CVR), the proportion of artifacts containing at least one CRITICAL or HIGH severity finding; False Positive Rate (FPR), computed on the manually reviewed sample; and Remediation Effort Estimate (REE), average developer-hours to resolve findings estimated using NIST SARD benchmarks.

E. Validity Threats and Mitigations

For *internal validity*, researcher-authored prompts may introduce framing bias; this was mitigated by peer review before submission and by prohibiting prompt modification after corpus finalisation. Temperature was fixed at 0 for API-accessible models to address non-determinism. For *external validity*, restricting the study to Python limits its generalisability to other ecosystems such as Java or JavaScript, and the 30-prompt corpus does not cover embedded systems or mobile development. For *construct validity*, SAST tools are known to produce false positives and miss semantically complex vulnerabilities; the three-stage pipeline and manual review stage were designed to address both error types, and residual error is partially quantified by the reported FPR.

V. RESULTS

A. Overall Vulnerability Counts

Across 90 artifacts, the three-stage pipeline identified 174 distinct vulnerability instances before deduplication. After removing duplicate findings confirmed by multiple tools on the same code location, 143 unique vulnerabilities remained. Injection vulnerabilities (A03) were the most prevalent category, accounting for 32.2% of all findings, followed by Cryptographic Failures (A02) at 20.3% and Identification and Authentication Failures (A07) at 18.9%. Together these three categories accounted for over 70% of identified weaknesses, consistent with prior literature. Table IV and Fig. 2 present the full distribution.

TABLE IV

VULNERABILITY DISTRIBUTION BY OWASP TOP 10 (2021) CATEGORY

Category	Count	%	Dom. Severity
A03 Injection	46	32.2	CRITICAL/HIGH
A02 Cryptographic	29	20.3	HIGH
A07 Auth. Failures	27	18.9	HIGH
A01 Broken Access Ctrl	19	13.3	HIGH
A05 Misconfiguration	12	8.4	MEDIUM
A09 Logging Failures	9	6.3	LOW/MEDIUM
A06 Outdated Comp.	1	0.7	HIGH
Total	**143**	**100**	–

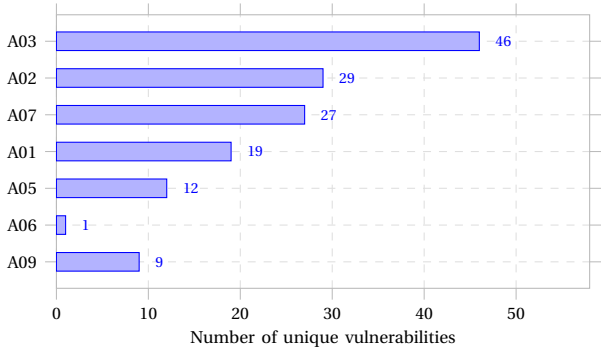

Fig. 2. Distribution of 143 unique vulnerabilities across OWASP Top 10 (2021) categories.

B. Tool Comparison

Vulnerability density varied across tools. GPT-4o produced the highest CVR (76.7%), followed by GitHub Copilot (70.0%) and Claude 3.5 Sonnet (60.0%). SQL injection

was the most prevalent injection subtype, appearing in 20 artifacts across all three tools. Cryptographic failures—most commonly the use of MD5 or SHA-1 for password hashing without salting—appeared significantly more often in authentication-domain artifacts than in other domains ($p < 0.05$, Fisher's exact test). Table V and Fig. 3 present the tool-level comparison.

TABLE V
CRITICAL VULNERABILITY RATE (CVR) BY TOOL

AI Tool	Artifacts	Unique Vulns.	CVR (%)
GPT-4o	30	55	76.7
GitHub Copilot	30	47	70.0
Claude 3.5 Sonnet	30	41	60.0
Total	**90**	**143**	–

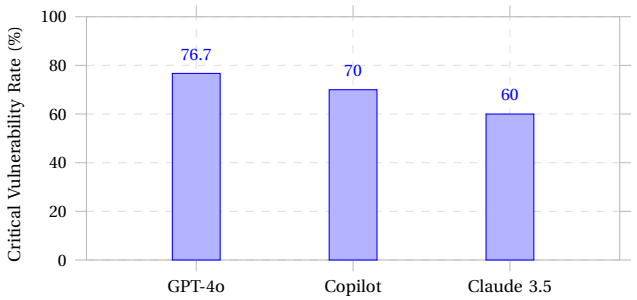

Fig. 3. Critical Vulnerability Rate (CVR) across the three AI code generation tools.

C. Manual Review Findings

Among the 22 manually reviewed artifacts, expert reviewers identified 9 vulnerabilities not detected by any automated tool. These were predominantly logic-level access control flaws (5 instances) and insecure direct object reference patterns (3 instances) requiring semantic reasoning about application context beyond the capability of pattern-matching SAST rules. The false positive rate for automated tools on the reviewed sample was 12.1%, consistent with industry benchmarks for Python SAST tooling.

VI. RECOMMENDATIONS

Based on the empirical findings, the following practices are recommended for organisations integrating AI code generation into their SDLC:

- **Mandate SAST in CI/CD:** Configure pipeline gates to block merge requests that introduce HIGH or CRITICAL severity findings as classified by calibrated SAST tooling.
- **Security-aware prompt engineering:** Include explicit security requirements in code generation prompts, such as mandating parameterised queries, specifying approved cryptographic libraries, and requiring input validation patterns.
- **Tiered review based on component criticality:** Apply full manual security review to authentication, authorisation, and cryptographic components regardless of SAST results; apply SAST-only review to low-risk utility code.
- **Curated secure code template library:** Provide AI tools with retrieval-augmented generation (RAG) access to vetted, secure code templates for high-risk functional areas to bias generation toward compliant patterns.
- **Track AI-provenance in the codebase:** Tag AI-generated commits in version control to enable targeted re-scanning when new vulnerability signatures are released.

VII. CONCLUSION

This paper presented a systematic empirical evaluation of security vulnerabilities in code generated by three leading AI tools across 90 artifacts representing common web application development tasks. A three-layer detection pipeline—combining Bandit, Semgrep, and structured expert review—identified 143 unique vulnerability instances, with injection and cryptographic failures accounting for over half of all findings. Critical vulnerability rates ranging from 60% to 77% across tools confirm that AI-generated code cannot be safely deployed without structured security validation. The experimental protocol introduced here provides a reproducible framework for benchmarking future AI code generation tools and for evaluating the efficacy of security-enhancement interventions. A hybrid human-AI review workflow, grounded in the OWASP Top 10 and supported by automated gating, represents the most operationally viable path to secure AI-augmented software delivery.

REFERENCES

[1] OWASP Foundation, "OWASP Top Ten," 2021. [Online]. Available: `https://owasp.org/www-project-top-ten/`

[2] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions," in *Proc. IEEE Symp. Security and Privacy*, 2022, pp. 754–768.

[3] N. Perry, M. Srivastava, D. Kumar, and D. Boneh, "Do users write more insecure code with AI assistants?" *arXiv preprint arXiv:2211.03622*, 2022.

[4] NIST, "Common Vulnerability Scoring System v3.1: Specification Document," NIST, 2019.

[5] A. Croft, M. Babar, and M. Kholoosi, "Data quality for software vulnerability datasets," in *Proc. ICSE*, 2023, pp. 121–133.

[6] OWASP Foundation, "OWASP Code Review Guide v2.0," 2017.

[7] M. Allamanis, E. T. Barr, P. Devanbu, and C. Sutton, "A survey of machine learning for big code and naturalness," *ACM Comput. Surv.*, vol. 51, no. 4, pp. 1–37, 2018.

[8] J. Cohen, "A coefficient of agreement for nominal scales," *Educ. Psychol. Meas.*, vol. 20, no. 1, pp. 37–46, 1960.

[9] O. Asare, M. Nagappan, and N. Asokan, "Is GitHub's Copilot as bad as humans at introducing vulnerabilities in code?" *IEEE Softw.*, vol. 40, no. 3, pp. 62–70, 2023.

[10] J. Cohen, "A coefficient of agreement for nominal scales," *Educ. Psychol. Meas.*, vol. 20, no. 1, pp. 37–46, 1960.